Choreographer

System Design Description

Abstract
This document describes the design and implementation of the Choreographer support system of Arrowhead (documentation edition 5.3.0) (`ah5-Choreographer-java-spring`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools . . . 4
3.2 Independent source and build ownership . . . 4
3.3 Functional properties implementation . . . 4
3.4 Non functional properties implementation . . . 5
3.5 Deployment . . . 5
3.6 Local modifications . . . 6
3.7 Verification . . . 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation . . . 7
5.2 Rust implementation . . . 8

6 References **8**

7 Revision History **10**
7.1 Amendments . . . 10
7.2 Quality Assurance . . . 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the Choreographer system implementation, which realizes the Choreographer SysD [1]. The Choreographer executes workflow plans: for each step it selects an executor system, has Dynamic ServiceOrchestration orchestrate the required provider for the executor, starts the executor and follows the results. In Section 3, we describe implementation details of the system.

3 Implementation

This implementation is based on the SysD document Choreographer [1] and is located in `ah5-Choreographer-java-spring`.

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9, Arrowhead common library 5.2.2 (private source under `vendor/`), Log4j 2; MariaDB Connector/J against MySQL 8.4 with TLS in the standalone deployment.
- Database `ah_choreographer` on MySQL 8.4 (MariaDB Connector/J, TLS with server verification in the standalone deployment), schema validated by Hibernate; SQL scripts in `src/main/resources/database`. Stateless request processing; scheduled jobs use Quartz.

3.2 Independent source and build ownership

The project folder owns its application source, Maven build, tests, executable JAR, Dockerfile, configuration and Compose definition. No source or image is taken from a sibling support folder or JavaSupportRuntime. Build with Java 21 and `mvn -B clean verify`; a populated external Maven dependency cache permits offline builds.

Arrowhead common utilities and DTOs are private source copies in `vendor/`, compiled into this system by the local Maven POM. No Arrowhead common/core parent or separately installed Arrowhead library is needed. Upstream POMs, licenses and source hashes are retained for provenance. Security updates must be propagated and tested in every affected copy. Third-party dependencies remain ordinary Maven libraries.

The packaged artifact is `arrowhead-choreographer-5.2.2.jar`. The image builds and tests this folder only; the system has a fixed application main class.

3.3 Functional properties implementation

The following components implement the system:

Component	Responsibility
`api.http`	Controllers of the four services and the orchestration-result endpoint.
`engine`	Session engine and step graph.
`service`	Plans, executors, sessions.
`jpa`	Entities.

Every request passes the common filter chain: authentication filter (`authentication.policy`), optional blacklist filter, management filter (management services), the system's authorization filter where present, then the controller, validation and normalization, the service logic and the database service.

3.3.1 Data handled and stored

Storage `ah_choreographer`:

Tables	Content
`plan, plan_action, plan_step, plan_step_connection`	Plans.
`executor, executor_service_definition`	Executors.

Tables	Content
`plan_session, session_step, worklog`	Execution.
`logs`	Log entries.

The data models of the interfaces are Java records (of `eu.arrowhead.dto` in the common library, or of the module); the IDD documents name them without the `DTO` suffix. The records used by this system are: `ActionRequestDTO`, `ActionResponseDTO`, `CheckPlanResponseDTO`, `ExecutedStepResultDTO`, `ExecutorCreateRequestDTO`, `ExecutorListResponseDTO`, `ExecutorRegistrationRequestDTO`, `ExecutorResponseDTO`, `ExecutorServiceDefinitionResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `OrchestrationServiceRequirementDTO`, `PageDTO`, `PlanListResponseDTO`, `PlanRequestDTO`, `PlanResponseDTO`, `RunPlanListRequestDTO`, `RunPlanListResponseDTO`, `RunPlanRequestDTO`, `RunPlanResponseDTO`, `SessionListResponseDTO`, `SessionResponseDTO`, `SessionStepListResponseDTO`, `SessionStepResponseDTO`, `StartSessionListRequestDTO`, `StartSessionRequestDTO`, `StepRequestDTO`, `StepResponseDTO`, `WorklogListResponseDTO`, `WorklogResponseDTO`.

3.4 Non functional properties implementation

3.4.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore, `client-auth=need`). The common library's filters implement the authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking.

3.4.2 Power management

None; the system is a server process without power states. The `sysMonitor power` observation always reports line supply.

3.4.3 Internal monitoring

Log4j 2 logs to stdout and files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The common library's `SysMonitorPublisher` publishes `echo`, `power` and `status` every 30 s.

3.4.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD; `generalManagement.get-config` returns the public ones.

3.5 Deployment

The local `compose.yaml` selects only this support system and its private MySQL database. It joins an operator-selected external cloud network; it does not start core systems or another support system. Copy `.env.example`, provision this system's own identity and credential files, validate the Compose definition, then run `docker compose up -d -build` from this folder. The image uses the secure external `config/application.properties`, mutual TLS, outsourced authentication, certificate/token binding and a mounted CRL. Secrets are read through Spring's file configuration tree. The app runs as UID 10001 with a read-only root filesystem and bounded non-executable temporary/log storage.

The database image initializes this system's own schema on an empty project-owned volume, reads independent random user/root passwords from files and uses a dedicated internal network. It has no host port. The application verifies the database TLS certificate against its public DB CA. The DB key and CA signing

key are never mounted into the application. Distinct Compose project names, host ports and private subnets are required for multiple deployments. Existing volumes require explicit migration/backup handling; ordinary shutdown preserves them.

Registry, Authentication, DSO, ConsumerAuthorization and required brokers/providers remain network dependencies. Independent deployment does not remove those service contracts. The listener health check is only a liveness check; inspect successful application startup, registration and authenticated service access before acceptance. The local `DEPLOYMENT.md` supplies the complete procedure. The previously recorded live-cloud deployment is historical baseline evidence; the new packaging does not change a live cloud.

3.6 Local modifications

Notes on this implementation in the workspace:

- Migration of v4.6; DSO push orchestration replaces orchestration by proxy.

3.7 Verification

Independent folder-only Maven builds run the project's JUnit suite. Artifact inspection checks the system main/source and absence of JavaSupportRuntime or other support-system implementations. Images and project-local Compose definitions are tested with private disposable state and generated test identities. Spring startup and authenticated HTTPS requests use isolated mutual-TLS core-service fixtures; missing-token and missing-certificate requests are refused. The distribution verification report records exact results and limitations. Broker, NTP and inter-cloud acceptance remain separate from deployment/startup validation.

4 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`choreographerSession`	[1]	[2]	[3]
`choreographerExecutor`	[1]	[4]	[5]
`choreographerManagement`	[1]	[6]	[7]
`generalManagement`	[1]	[8]	[9]
`sysMonitor`	[1]	[10]	[11]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[12]	[13]	–
identity	[14]	[15]	–
authorization	[16]	[17]	–
serviceOrchestrationPush Management	[18]	[19]	–
executor interface	Executors	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-Choreographer-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8457 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8457"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Plan DAG validation, executor registration, synchronous orchestration, persistent steps/callbacks/worklog. Async result callback (explicit 501), complete dependency negotiation, optimal executor selection, session notification, remote abort propagation, inter-cloud sessions. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-Choreographer-go/documentation/Choreographer`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-Choreographer-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8457 (Arrowhead default, role choreographer). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Plan/executor CRUD, single local action/step starts, aborts and completion callbacks. Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Choreographer`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] J. Delsing, “Choreographer – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-support-java-spring/Choreography/documentation/v5.3.0/Choreographer-v5.3.0-SysD.pdf.

[2] ——, “choreographerSession – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.3.0, 2026, choreographer-v5.3.0-choreographerSession-SD.pdf.

[3] ——, “choreographerSession HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-choreographerSession-HTTP-IDD.pdf.

[4] ——, “choreographerExecutor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.3.0, 2026, choreographer-v5.3.0-choreographerExecutor-SD.pdf.

[5] ——, “choreographerExecutor HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-choreographerExecutor-HTTP-IDD.pdf.

[6] ——, “choreographerManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.3.0, 2026, choreographer-v5.3.0-choreographerManagement-SD.pdf.

[7] ——, “choreographerManagement HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-choreographerManagement-HTTP-IDD.pdf.

[8] ——, “generalManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.3.0, 2026, choreographer-v5.3.0-generalManagement-SD.pdf.

[9] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-generalManagement-HTTP-IDD.pdf.

[10] ——, “sysMonitor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.3.0, 2026, choreographer-v5.3.0-sysMonitor-SD.pdf.

[11] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.3.0, 2026, choreographer-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[12] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[13] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-systemDiscovery-SD.pdf.

[14] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[15] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-identity-SD.pdf.

[16] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-SysD.pdf.

[17] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-authorization-SD.pdf.

[18] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.3.0/DynamicServiceOrchestration-v5.3.0-SysD.pdf.

[19] ——, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.3.0/DynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Self-contained build, private library source, configuration, images, database and isolated deployment verification.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	